

Design and Implementation of RISC Processor using Verilog



A

Project Report

Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS ENGINEERING - VLSI (DESIGN AND TECHNOLOGY)

Submitted by

Group members

Vishakha vasishta - 2317103

Ishita - 2317062

Manokamna - 2317076

Garima Chaudhary - 2317058

Under the guidance of

Dr. Shekhar Yadav

Department of Physical Sciences

Banasthali Vidyapith

Rajasthan, India

(2026)

Certificate

This is to certify that that the work, which is being presented in the Project, entitled the “Design and Implementation of RISC Processor using Verilog” in partial fulfillment for the award of Degree of “Bachelor of Technology” in Deptt. of Electronics with Specialization in VLSI Design and Technology, and submitted to the Department of Electronics, Banasthali Vidyapith, Rajasthan

It is a record of my own investigations carried under the Guidance of Mr./ Dr. Shekhar Yadav , Department of Electronics, Banasthali Vidyapith.

I have not submitted the matter presented in this report anywhere for the award of any other Degree.

(Vishakha Vashishtha - 2317103,

Ishita - 2317062,

Manokamna - 2317076,

Garima Chaudhary - 2317058)

VLSI Design and Technology,

Banasthali Vidyapith, Rajasthan

Acknowledgement

We would like to sincerely thank our guide for their constant support and valuable guidance throughout the whole project. Their suggestions and feedback helped us understand the concepts more clearly and improve our work at every stage.

We are also grateful to our institution for providing us with the required facilities and a learning environment, which made it is easier to complete this project successfully. The academic support and facilities offered have greatly contributed to the smooth execution of this work.

Lastly, we thank everyone who directly or indirectly supported us during this work. Their help is deeply appreciated.

.

List of Content

1. Abstract
2. Introduction
3. Literature Survey
4. Objectives
5. Methodology
6. Working Principle
7. Module design and simulation
 - Instruction Fetch (IF) Module
 - Instruction Memory Module
 - IF-ID Pipeline Register Module
 - Register File Module
 - Arithmetic Logic Unit (ALU) Module
 - Control Unit Module
8. Overall Architecture of the processor
9. Results and Simulation
10. Conclusion
11. Future Scope
12. References

Abstract

This project presents the design and implementation of a basic Reduced Instruction Set Computing (RISC) processor using the Verilog Hardware Description Language (HDL). The primary objective of this work is to develop a functional processor model based on RISC principles, emphasizing simplicity, modularity, and efficient instruction execution.

The processor is developed using a modular approach, where different parts such as Instruction Fetch, Decode, Execute, Memory, and Write Back stages are designed separately and then connected together. A pipelined architecture is used so that multiple instructions can be processed at the same time, which improves performance and efficiency.

Special focus is given to important components like the ALU, which performs calculations, and the Register File, which stores data. The control unit generates signals to manage all operations properly. Basic pipeline issues are also considered to ensure smooth data flow.

The design is tested using simulation tools like Vivado and EDA Playground. Functional simulation is carried out to validate the correctness of instruction flow, timing, and overall processor behavior. The results obtained from the simulations confirm that the processor executes instructions accurately and efficiently, demonstrating the effectiveness of the RISC architecture and the reliability of the proposed design.

Overall, this project helps in understanding processor design practically and strengthens knowledge of digital systems and computer architecture

Introduction

Reduced Instruction Set Computer (RISC) architecture is a design philosophy in computer architecture that focuses on simplifying instruction execution to achieve higher performance and efficiency. Unlike Complex Instruction Set Computer (CISC) architectures, which utilize a large and complex set of instructions, RISC architectures are based on a smaller, highly optimized set of instructions that are typically executed within a single clock cycle. This simplification enables faster instruction processing, easier hardware implementation, and improved pipeline efficiency.

The concept of RISC architecture is based on the use of simple instructions, a load/store memory approach, and a large number of general-purpose registers. In this approach, memory access is restricted to specific load and store instructions, while all other operations are performed using registers. This reduces memory dependency and enhances the execution speed. Additionally, the uniform instruction format and fixed instruction length make decoding simpler and more efficient.

In this project we focus on the design and implementation of a basic RISC processor using Verilog Hardware Description Language (HDL). The primary aim is to gain a clear understanding of how a processor operates at the hardware level and how different functional units interact to execute instructions. The processor is developed using a modular design methodology, where each component—such as the Instruction Fetch unit, Instruction Decode unit, Register File, Arithmetic Logic Unit (ALU), Memory unit, and control logic—is designed and tested individually before being integrated into a complete system.

Through this work, we gained practical experience in Verilog coding, simulation, and debugging using tools like Vivado. This project helps in connecting theoretical concepts with real implementation and builds a strong base for further studies in processor design and VLSI.

Furthermore, This not only strengthens theoretical knowledge but also develops essential skills in hardware modeling, debugging, and system-level design.

Overall, the project serves as a foundational study of processor architecture and digital system design, bridging the gap between theoretical concepts and practical implementation. It enhances understanding of modern processor design techniques and prepares students for more advanced topics in computer architecture and VLSI design.

Literature Survey

The field of processor design has evolved significantly over the years, with Reduced Instruction Set Computer (RISC) architecture playing an important role in modern computing systems [\[2\]](#)[\[5\]](#). Early research in computer architecture emphasized simplifying instruction sets to improve execution speed and hardware efficiency. This led to the development of RISC-based systems, which focus on executing simple instructions within a single clock cycle and utilizing efficient pipelining techniques [\[3\]](#).

One of the major contributions to this field is the development of the RISC-V architecture, an open-standard Instruction Set Architecture (ISA) that has gained widespread adoption in both academia and industry [\[4\]](#)[\[6\]](#). Its modular design makes it highly suitable for educational purposes and custom processor implementations. Several studies highlight how RISC enables designers to implement scalable and flexible processors while maintaining simplicity in hardware design [\[2\]](#)[\[3\]](#).

Previous studies have shown how basic RISC processors can be designed using hardware description languages like Verilog and VHDL [\[1\]](#)[\[7\]](#). These designs usually include important components such as ALU, register file, control unit, and memory. Many of them also use pipelining to improve performance by executing multiple instructions at the same time [\[3\]](#).

Literature also emphasizes the importance of pipeline design in enhancing processor performance. The standard 5-stage pipeline—Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back—has been widely studied and implemented due to its balanced trade-off between complexity and performance [\[3\]](#). Researchers have explored various techniques to handle pipeline hazards, such as data forwarding, stalling, and branch prediction, to ensure correct and efficient instruction execution [\[3\]](#).

Simulation and verification play a vital role in validating processor designs. Prior works have utilized tools such as Xilinx Vivado, ModelSim, and online simulation platforms to test functionality, timing, and correctness of processor modules [\[1\]](#)[\[8\]](#). These tools enable designers to identify errors, optimize performance, and ensure reliable operation before hardware implementation.

Overall, past research provides a strong base for this project. The concepts of modular design, pipelining, and simulation used in this project are based on these earlier works. This work builds upon these established principles to design and simulate a functional RISC processor.

Objectives

The primary objective of this project is to design and implement an efficient and scalable Reduced Instruction Set Computer (RISC) processor using Verilog Hardware Description Language (HDL), with a focus on understanding advanced concepts of computer architecture and pipelined processor design.

The specific objectives of this project are as follows:

- To analyze and apply the fundamental principles of RISC architecture, emphasizing simplicity, modularity, and high-performance instruction execution.
- To design and implement a 5-stage pipelined processor architecture, enabling parallel execution of instructions to improve throughput and overall system efficiency.
- To develop a complete processor datapath, including key components such as the Program Counter, Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Data Memory, and Control Unit.
- To design an efficient control mechanism for accurate instruction decoding and generation of control signals for different instruction formats.
- To implement pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) and ensure proper synchronization and data flow across all stages.
- To perform functional verification and timing analysis of the processor design using simulation tools like Vivado and EDA Playground.
- To evaluate the performance improvements achieved through pipelining by comparing it with a non-pipelined architecture.
- To enhance practical skills in Verilog HDL coding, simulation, debugging, and digital system design methodologies.

This project ultimately aims to provide in-depth knowledge of processor design at the hardware level and to prepare for more advanced studies and applications in VLSI design and embedded systems.

Methodology

The methodology adopted for this project follows a systematic and modular approach to design and implement a RISC processor using Verilog Hardware Description Language (HDL). The development process is divided into multiple stages, ensuring clarity, scalability, and ease of debugging.

Initially, the processor architecture is defined based on the principles of Reduced Instruction Set Computer (RISC). The instruction set, data path, and control requirements are identified to establish a clear design framework. A 5-stage pipelined architecture is selected to improve instruction throughput and overall system performance.

The next step involves the design of individual functional modules. Each component of the processor—such as the Instruction Memory, Program Counter, Instruction Fetch unit, Instruction Decode unit, Register File, Arithmetic Logic Unit (ALU), Data Memory, and Control Unit—is designed separately using Verilog. The modular design approach allows independent development and testing of each unit, reducing complexity and improving reliability.

Once the individual modules are implemented, pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) are introduced to enable pipelined execution. These registers ensure proper data transfer between stages and maintain synchronization across clock cycles.

After module design, integration of all components is carried out to form the complete processor data path. Control signals are carefully designed and connected to ensure correct instruction decoding and execution. Special attention is given to timing and synchronization to avoid errors during data flow.

To ensure correct processor functionality, simulation and verification are performed using tools such as Vivado and EDA Playground. Various test cases and instruction sequences are applied to validate arithmetic operations, memory access, and instruction flow. Debugging is performed to resolve issues related to control flow, and signal mismatches.

Finally, the performance of the processor is analyzed based on correctness of output, instruction execution flow, and pipeline efficiency. The modular and pipelined design demonstrates improved performance compared to a non-pipelined approach, validating the effectiveness of the chosen methodology.

Working Principle

The proposed RISC processor operates based on a structured sequence of pipeline stages, enabling efficient execution of instructions through parallel processing. The overall execution is divided into five distinct stages, namely Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage performs a specific function and operates concurrently with other stages, thereby enhancing the throughput of the processor.

In the Instruction Fetch (IF) stage, the Program Counter (PC) holds the address of the next instruction to be executed. This address is used to fetch the instruction from the instruction memory. After fetching, the Program Counter is updated to point to the subsequent instruction, ensuring continuous instruction flow.

The fetched instruction is then transferred to the Instruction Decode (ID) stage, where it is decoded by the control unit to determine the type of operation to be performed. During this stage, the instruction fields are analyzed to identify source and destination registers. Accordingly, the register file reads the required operands from the specified registers.

Subsequently, the instruction proceeds to the Execute (EX) stage, where the Arithmetic Logic Unit (ALU) performs the required arithmetic or logical operations based on the control signals generated during the decoding stage. This stage is responsible for computations such as addition, subtraction, logical AND/OR, and comparison operations.

In the Memory Access (MEM) stage, memory-related operations are carried out. If the instruction involves data transfer, such as load or store operations, the processor accesses the data memory to either read data from memory or write data into memory. For instructions that do not require memory access, this stage is effectively bypassed.

Finally, in the Write Back (WB) stage, the result obtained from the ALU or memory is written back into the destination register in the register file. This completes the execution of the instruction.

The pipelined architecture allows multiple instructions to be processed simultaneously at different stages of execution. As a result, during each clock cycle, a new instruction enters the pipeline while previous instructions advance to subsequent stages. This overlapping of instruction execution significantly improves the overall performance and efficiency of the processor compared to a non-pipelined design.

Module Design and Simulation

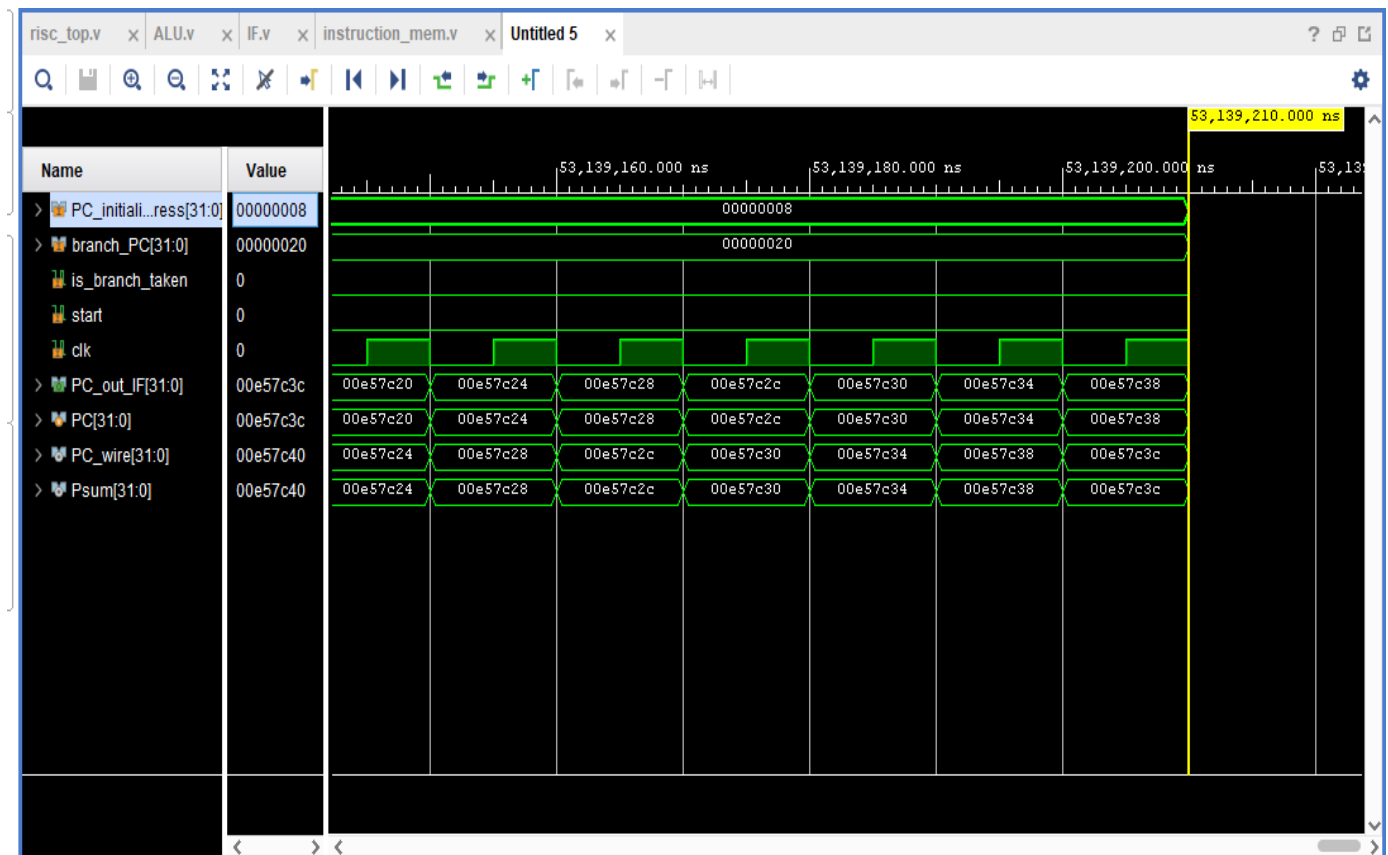
• Instruction Fetch (IF) Module

The Instruction Fetch (IF) module is the first stage of the pipelined RISC processor. Its primary function is to fetch the next instruction from instruction memory using the Program Counter (PC).

At every clock cycle, the PC holds the address of the current instruction. This address is sent to the instruction memory, which outputs the corresponding instruction. Simultaneously, an adder computes the next sequential address by adding 4 to the current PC value ($PC + 4$), since each instruction is 32 bits (4 bytes) long.

The next value of the PC is selected based on the branch condition. If a branch is not taken, the PC is updated with $PC + 4$. If a branch is taken, the PC is updated with the branch target address. This selection is controlled by the branch signal.

Thus, the IF module ensures continuous and correct instruction fetching by updating the PC and supplying instructions to the next pipeline stage.



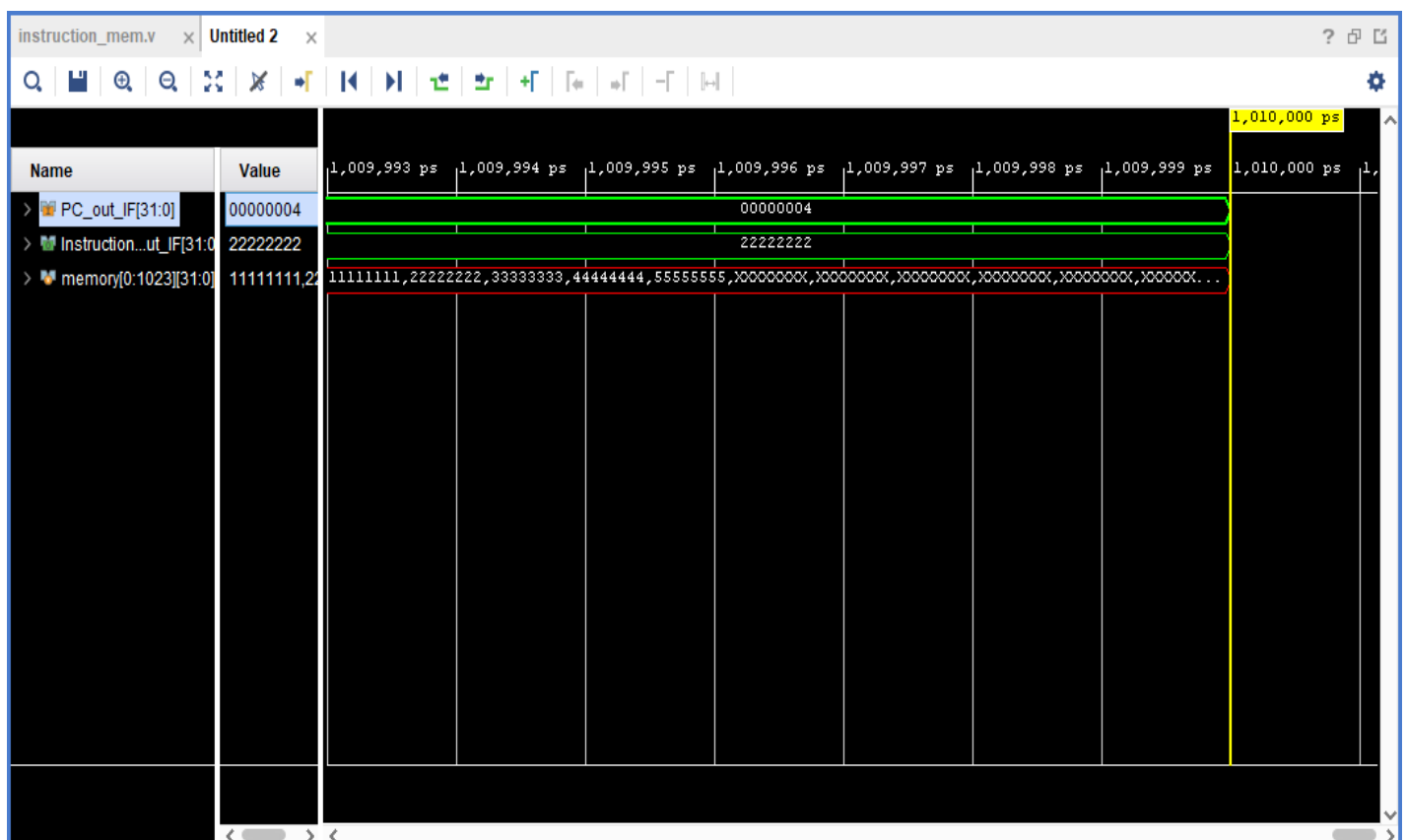
- **Instruction Memory Module**

The Instruction Memory module is responsible for storing the program instructions and providing them to the processor during execution. It acts as a read-only memory where instructions are preloaded before execution begins.

The module takes the instruction address, usually provided by the Program Counter (PC), as input. Based on this address, it outputs the corresponding instruction stored at that memory location. Each instruction is typically 32 bits in size.

The memory is organized such that each address corresponds to a specific instruction. As the PC updates sequentially (or changes due to branching), the instruction memory continuously supplies the correct instruction to the Instruction Fetch (IF) stage.

Since instruction memory is read-only during execution, no write operations are performed, ensuring stable and consistent instruction flow throughout the processor operation.



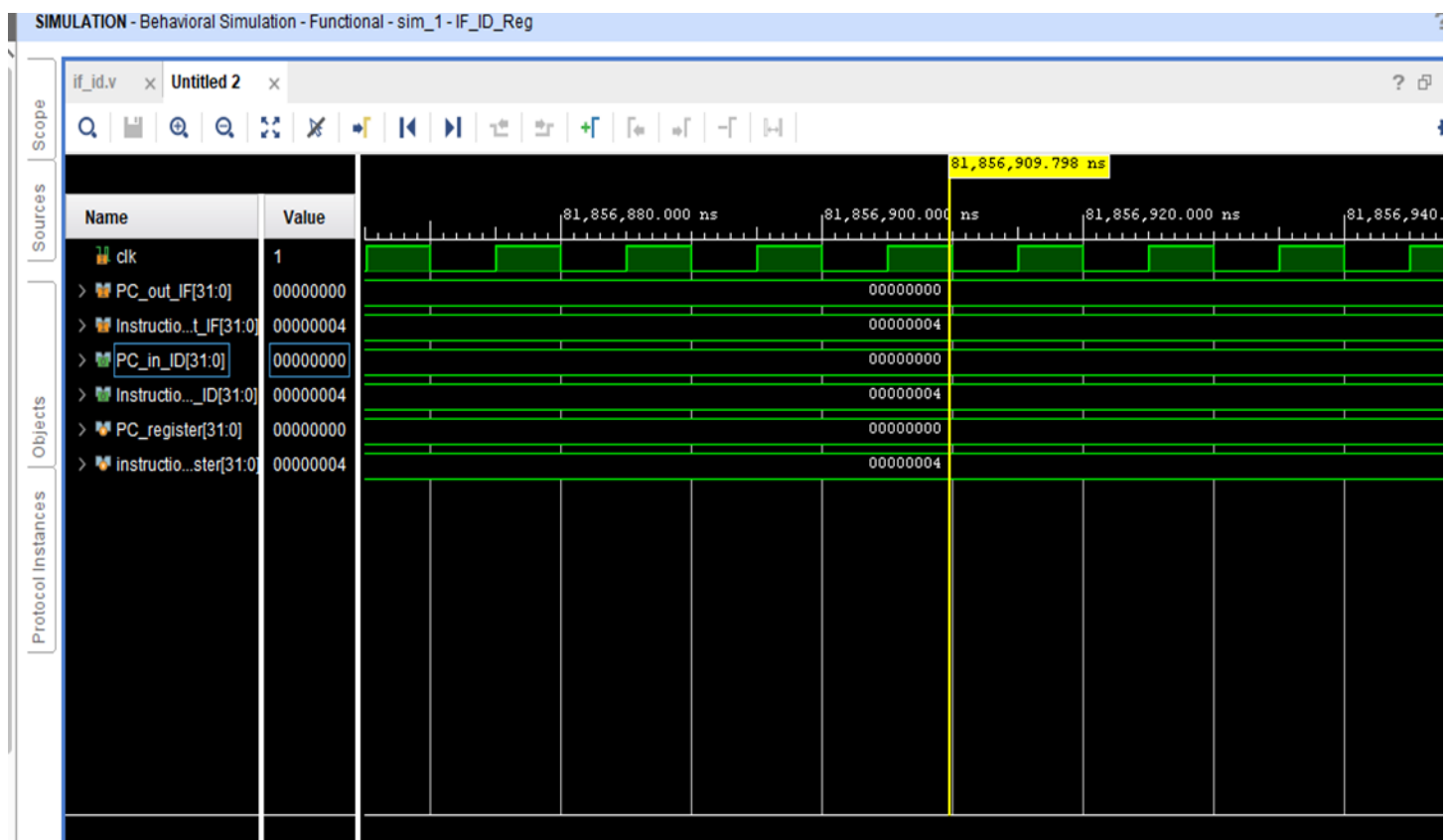
- **IF-ID Pipeline Register Module**

The IF-ID (Instruction Fetch to Instruction Decode) pipeline register acts as an intermediate storage element between the Instruction Fetch (IF) stage and the Instruction Decode (ID) stage in a pipelined processor.

Its primary function is to hold the fetched instruction and the corresponding Program Counter (PC) value temporarily, so that they can be used by the next stage in the following clock cycle. This ensures proper synchronization and smooth data transfer between pipeline stages.

At every clock edge, the IF-ID register captures the outputs of the IF stage, including the instruction and PC value, and stores them internally. These stored values are then forwarded to the Instruction Decode stage for further processing, such as decoding and operand fetching.

By introducing this pipeline register, multiple instructions can be processed simultaneously in different stages of the pipeline, thereby improving overall processor performance and throughput.

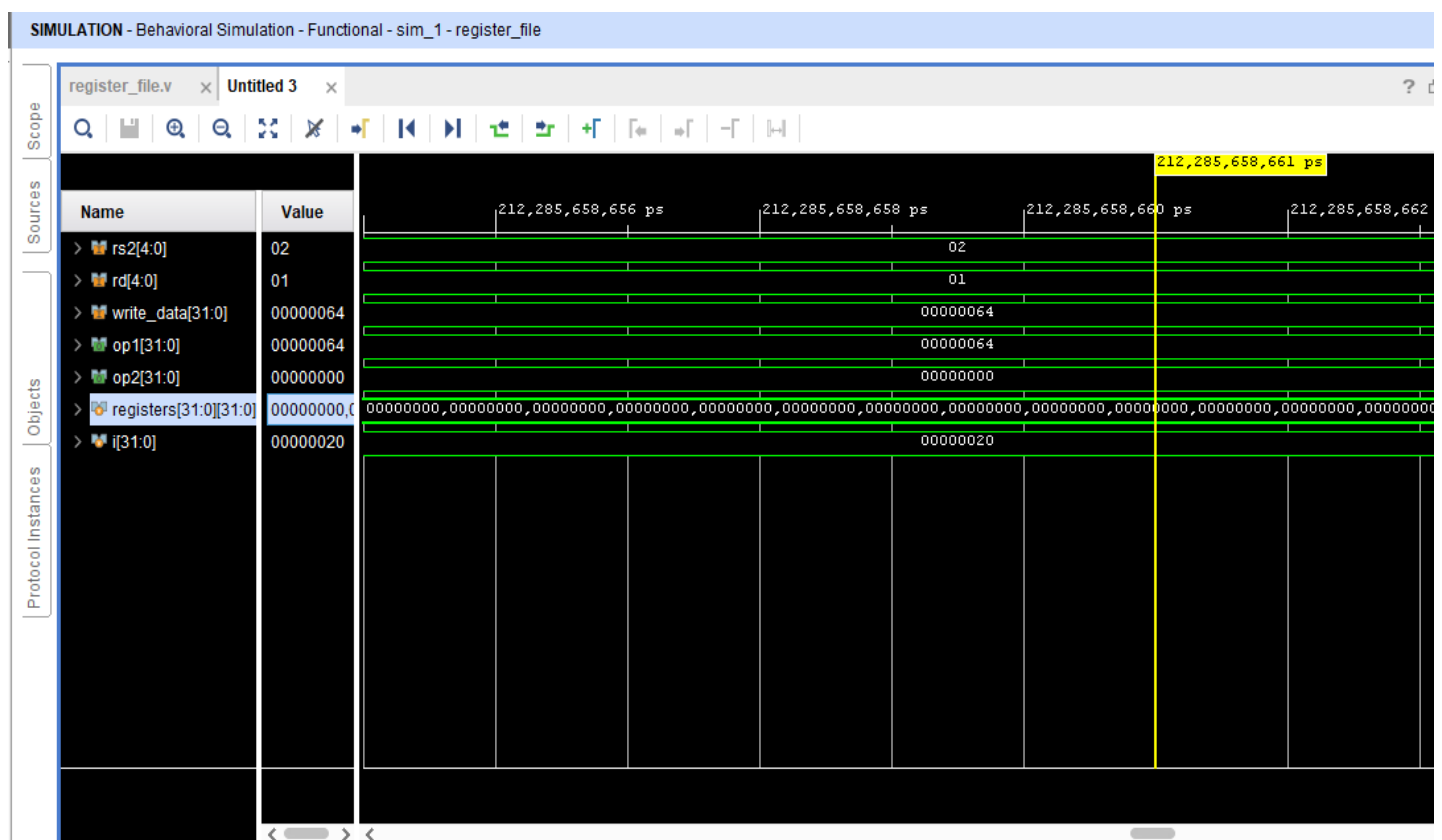


- **Register File Module**

The Register File is a key component of the RISC processor that is used to store and provide operands for instruction execution. It consists of a set of general-purpose registers that can be accessed quickly during processor operation.

The module supports two read operations and one write operation simultaneously. It takes two read register addresses as inputs and outputs the corresponding data stored in those registers. These outputs are used as operands for the Arithmetic Logic Unit (ALU).

For write operations, the module takes a destination register address along with the data to be written. When the write enable signal (RegWrite) is active, the data is written into the specified register during the clock cycle. This design allows efficient data access and supports parallel read operations, which is essential for high-performance pipelined execution in RISC architectures.



- **Arithmetic Logic Unit (ALU) Module**

The Arithmetic Logic Unit (ALU) is a fundamental component of the processor responsible for performing arithmetic and logical operations on input operands. It receives two input values from the register file along with an ALU control signal that determines the operation to be executed.

Based on the control signal, the ALU can perform a variety of operations including addition, subtraction, multiplication, division, logical AND, logical OR, and logical NOT. Each operation is selected using a specific control code, enabling the ALU to handle multiple instruction types efficiently.

The ALU operates as a combinational circuit, producing outputs based on the current inputs without requiring clocked storage. The result generated by the ALU is then forwarded to subsequent stages such as memory access or write-back.

Additionally, the ALU may generate status signals such as the zero flag, which is useful in conditional branch operations.

- **Control Unit Module**

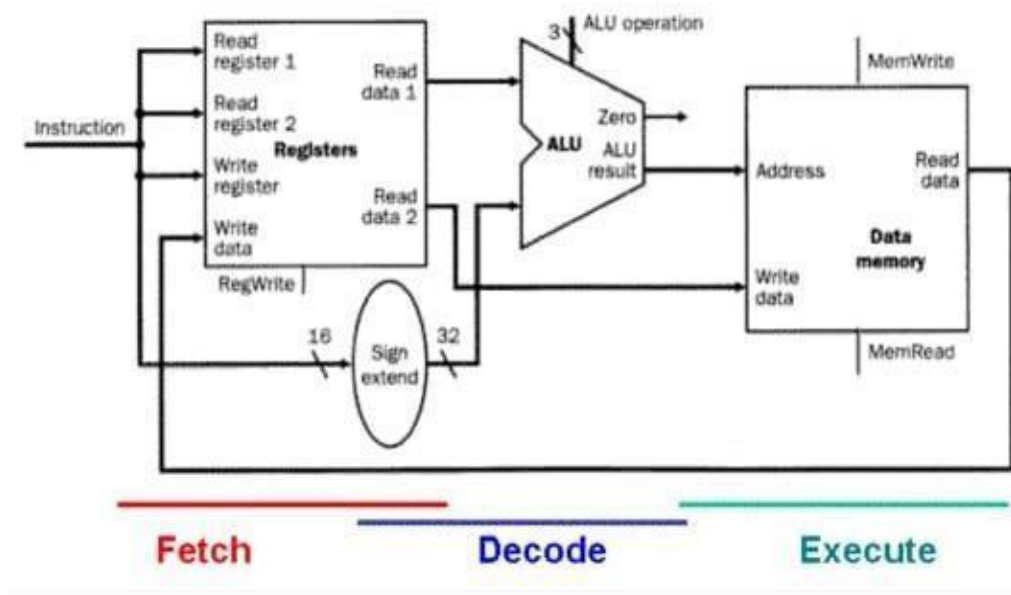
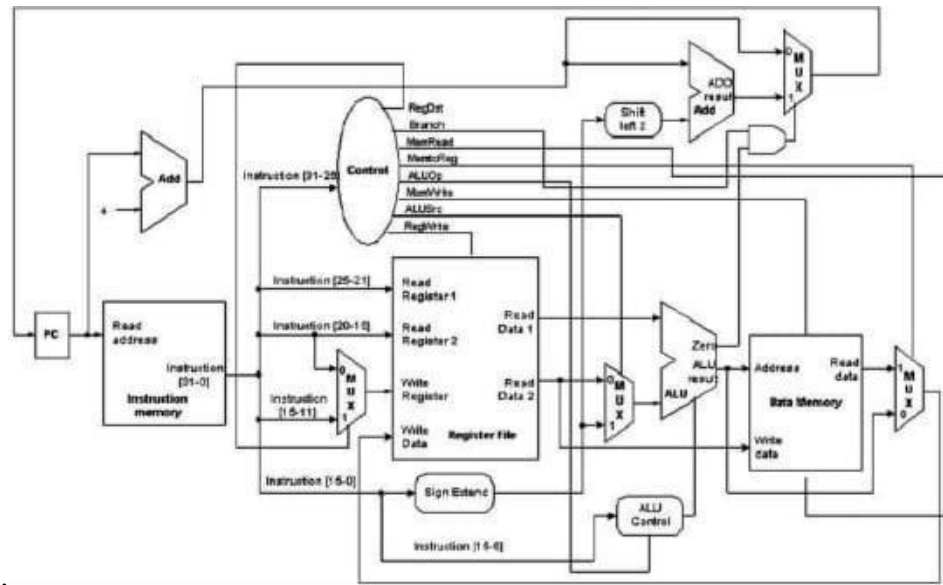
The Control Unit is responsible for generating the control signals required to coordinate the operation of different modules in the processor. It takes the instruction opcode (and sometimes function fields) as input and produces appropriate control signals for execution.

Based on the instruction type, the control unit activates signals such as RegWrite, MemRead, MemWrite. These signals control the flow of data between the register file, ALU, and memory, ensuring correct instruction execution.

The control unit essentially acts as the brain of the processor, directing each stage of the pipeline to perform the required operations. It ensures that the correct sequence of actions is followed for each instruction.

In a pipelined architecture, the control signals are generated in the decode stage and propagated through the pipeline registers to control operations in later stages.

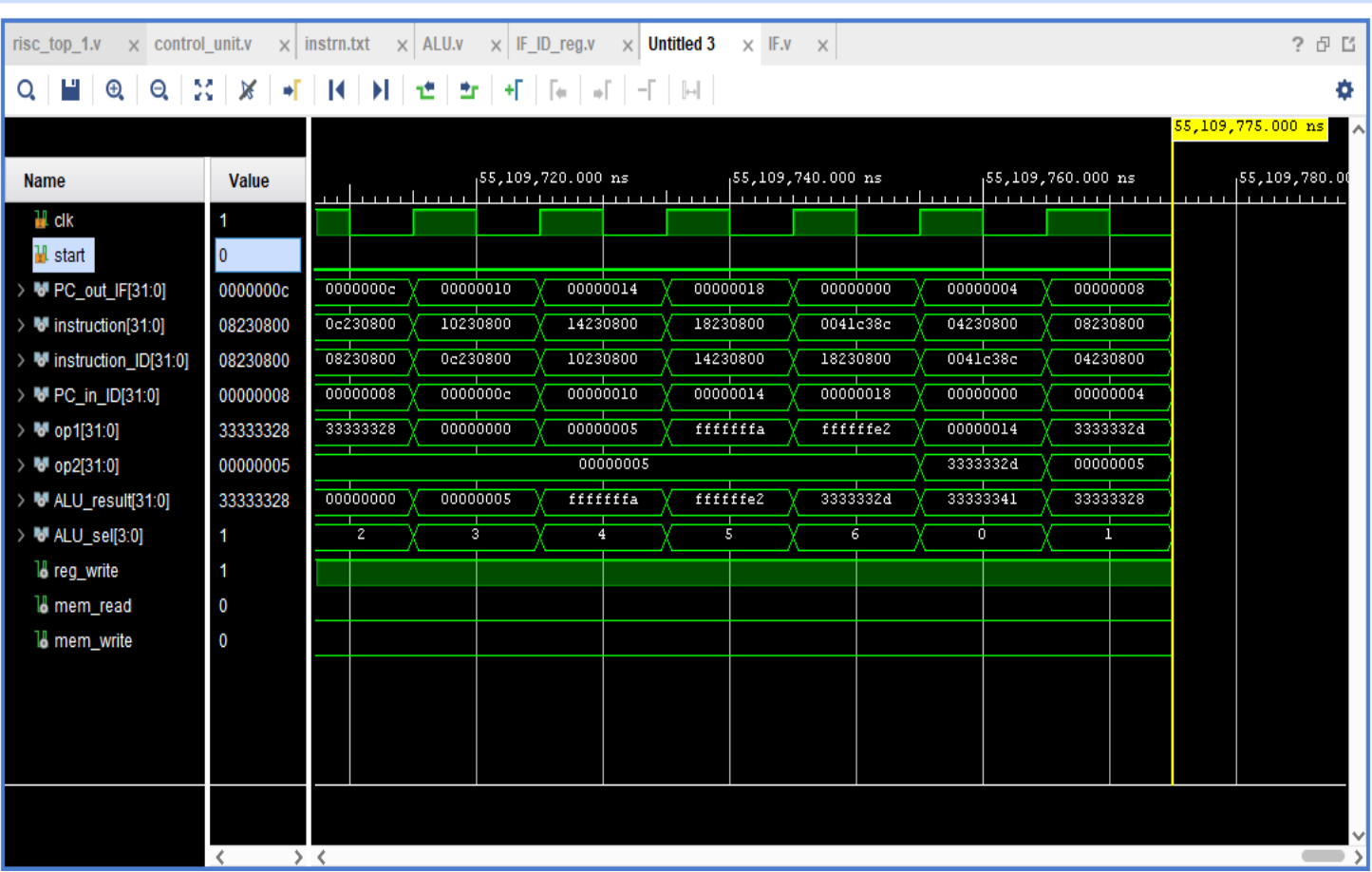
Overall Architecture of the Processor



RISC Processor Architecture

Results and Simulation

The simulation results confirm that the processor is functioning correctly. The program counter increments as expected, and instructions are fetched sequentially. The register file provides correct operand values, and the ALU produces accurate results. Waveforms obtained from simulation tools clearly show the data flow and timing behavior of the processor.



Simulation waveform of RISC Processor

Conclusion

The project successfully demonstrates the design and implementation of a basic RISC processor using a pipelined architecture. It highlights the significance of a modular design approach, where individual components such as the Program Counter, Register File, ALU, Control Unit, and pipeline registers are designed and integrated systematically. This modular methodology simplifies both the development and debugging processes in complex digital systems.

The simulation results obtained from the implemented design validate the correct functionality of the processor. The waveform analysis clearly verifies that the processor operates according to the expected model, with instructions flowing correctly through each pipeline stage and producing accurate outputs. The use of user-defined input data further confirms the correctness and reliability of the implemented design.

Additionally, this project incorporates several fundamental concepts essential for processor design, including instruction execution, pipelining, data flow management, and control signal generation. These concepts collectively provide a comprehensive understanding of how a processor operates at a structural and functional level.

Overall, this project serves as a strong foundation for exploring advanced topics in processor design, such as hazard detection, forwarding techniques, cache memory integration, and performance optimization. It provides valuable practical insight into modern CPU design methodologies and prepares the groundwork for future enhancements and research in the field of computer architecture.

Future Scope

The proposed processor design can be further enhanced by incorporating a more comprehensive **control unit** capable of supporting a wider range of instructions. This would improve the functionality and flexibility of the processor. Additionally, advanced **pipelining techniques** can be implemented to further enhance performance by increasing instruction throughput.

To improve execution efficiency, **hazard detection and forwarding units** can be integrated into the design. These mechanisms will help in minimizing pipeline stalls and resolving data dependencies, thereby optimizing overall processor performance.

Moreover, the processor can be implemented on **FPGA (Field Programmable Gate Array)** platforms for real-time testing and validation. This would provide practical insights into hardware-level behavior and performance under real operating conditions.

Looking ahead, the relevance of such processor designs is expected to grow significantly due to advancements in key industries and emerging technologies. Fields such as **Artificial Intelligence (AI)**, data centers, virtual currency computing, automotive systems, communication technologies, and green energy solutions, including electric vehicles (EVs) and charging infrastructure, are anticipated to drive future innovation.

At the core of this technological transformation lies the semiconductor industry, where **RISC architecture** is emerging as a crucial enabler of innovation, flexibility, and sustainability. Its open-source nature and scalability make it highly suitable for next-generation applications, positioning it as a cornerstone for future processor development.

References

- [1] Morris Mano, Computer System Architecture. Vivado Design Suite Documentation. EDA Playground Online Simulation Tool. Various online tutorials and academic resources on RISC architecture and Verilog HDL.
- [2] David A. Patterson & John L. Hennessy (2017), Computer Architecture: A Quantitative Approach (6th ed.), Morgan Kaufmann.
- [3] David A. Patterson & John L. Hennessy (2017), Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, Morgan Kaufmann.
- [4] RISC-V Official Website, “Outlook 2025: The Role of RISC-V in Shaping the Future.”
<https://riscv.org/blog/outlook-2025-the-role-of-risc-v-in-shaping-the-future/>
- [5] Patterson, D. A., & Waterman, A. (2017), The RISC-V Reader: An Open Architecture Atlas, Strawberry Canyon LLC.
- [6] RISC-V Foundation, “RISC-V Specifications.”
<https://riscv.org/technical/specifications/>
- [7] M. Morris Mano & M. D. Ciletti, Digital Design with Verilog HDL, Pearson.
- [8] Xilinx Inc., Vivado Design Suite User Guide.
<https://www.xilinx.com>